جامعة سيدي محمد بن عبد الله بفاس
UNIVERSITÉ SIDI MOHAMED BEN ABDELLAH DE FES

كلية العلوم ظهر المهراز فاس
FACULTE DES SCIENCES DHAR EL MAHRAZ FES

Département Informatique

Master Machine Learning Avancé et Intelligence Multimédia
Parcours : Intelligence Multimédia

CONCIERGE VOYAGE MCP

Plateforme Intelligente de Planification de Voyage

Réalisé par :

AZZOUZI wassima

Encadré par :

M. Jamal RIFFI

Année universitaire 2025-2026

Table des matières

1 Introduction et Problématique **3**

2 Solution Proposée : Concierge Voyage MCP **3**
2.1 Fonctionnalités Centrales ... 3
2.2 Innovation Architecturale ... 3

3 Axe 1 : Fonctionnement du Système **4**
3.1 Architecture Globale ... 4
3.1.1 Description des Couches ... 4
3.2 Flux de Données - Recherche de Vols ... 5
3.2.1 Séquence Détaillée ... 6
3.3 Flux de Données - Calcul d'Itinéraire ... 7
3.3.1 Processus Technique ... 7
3.4 Interactions entre Composants ... 8

4 Axe 2 : Interfaces Utilisateur **8**
4.1 Onglet "Recherche de Vols" () ... 9
4.1.1 Composants Clés ... 9
4.2 Onglet "Météo & Infos" () ... 10
4.2.1 Architecture ... 10
4.2.2 Intelligence Contextuelle ... 10
4.3 Onglet "Trajets & Maps" () ... 10
4.3.1 Fonctionnalités Avancées ... 10
4.4 Architecture Technique du Frontend ... 11
4.4.1 Gestion d'État ... 11
4.4.2 Responsive Design ... 11

5 Axe 3 : Technologies et Concepts Architecturaux **12**
5.1 Stack Technique Complet ... 13
5.2 Pattern MCP (Model Context Protocol) ... 13
5.2.1 Structure du Serveur MCP ... 13
5.2.2 Avantages du Pattern MCP ... 14
5.3 APIs Externes Intégrées ... 14
5.3.1 Amadeus API (Vols) ... 14
5.3.2 OpenWeather API (Météo) ... 14
5.3.3 OpenRouteService (Itinéraires) ... 14
5.3.4 Overpass API (POIs) ... 14
5.4 Concepts Architecturaux Clés ... 14
5.4.1 Session State Management ... 14
5.4.2 Pattern Fallback ... 15
5.4.3 Composition de Données ... 15
5.5 Gestion des Erreurs et Sécurité ... 15
5.5.1 Stratégies de Gestion d'Erreurs ... 15
5.5.2 Sécurité ... 15

6 Conclusion **15**
6.1 Forces Techniques 15
6.2 Perspectives d'Évolution 16

Annexes **17**

1 Introduction et Problématique

Dans l'écosystème contemporain du voyage, les voyageurs font face à une **fragmentation croissante des outils digitaux** : ils doivent consulter des comparateurs de vols distincts, des applications météo séparées, des services de cartographie différents et des plateformes de réservation hôtelière multiples pour planifier un simple trajet. Cette dispersion informationnelle engendre une **perte de temps considérable**, des **incohérences de données** (horaires non synchronisés, informations obsolètes) et une **expérience utilisateur dégradée** caractérisée par des allers-retours constants entre applications.

Par ailleurs, l'émergence des assistants IA (Claude, Gemini) soulève un défi architectural majeur : comment permettre à une intelligence artificielle d'interagir de manière standardisée avec des services externes complexes (réservation, géolocalisation, météo) sans développer des connecteurs spécifiques pour chaque cas d'usage ?

La problématique centrale réside dans la conception d'une **architecture unifiée, modulaire et interopérable** capable d'agréger en temps réel des données hétérogènes (vols, météo, itinéraires routiers, points d'intérêt) tout en restant accessible tant aux humains via une interface graphique qu'aux machines via un protocole standar- disé.

2 Solution Proposée : Concierge Voyage MCP

Concierge Voyage MCP est une application web de planification de voyage construite sur une **architecture hybride** combinant une interface utilisateur Streamlit intuitive et un serveur MCP (Model Context Protocol) permettant l'intégration avec des assistants IA.

2.1 Fonctionnalités Centrales

Le système centralise quatre fonctionnalités critiques :

- **Recherche de vols** en temps réel via l'API Amadeus
- **Prévisions météorologiques** via OpenWeather
- **Calcul d'itinéraires routiers** via OpenRouteService
- **Découverte de services à proximité** (hôtels, restaurants) via l'API Overpass d'OpenStreetMap

2.2 Innovation Architecturale

L'innovation majeure réside dans l'adoption du **pattern MCP** qui standardise l'interface entre le modèle de langage et les outils externes, transformant chaque fonctionnalité en un *"outil"* autonome et réutilisable. Cette approche découple complètement la logique métier de l'interface utilisateur.

3 Axe 1 : Fonctionnement du Système

3.1 Architecture Globale

Le système adopte une **architecture en quatre couches** strictement séparées :

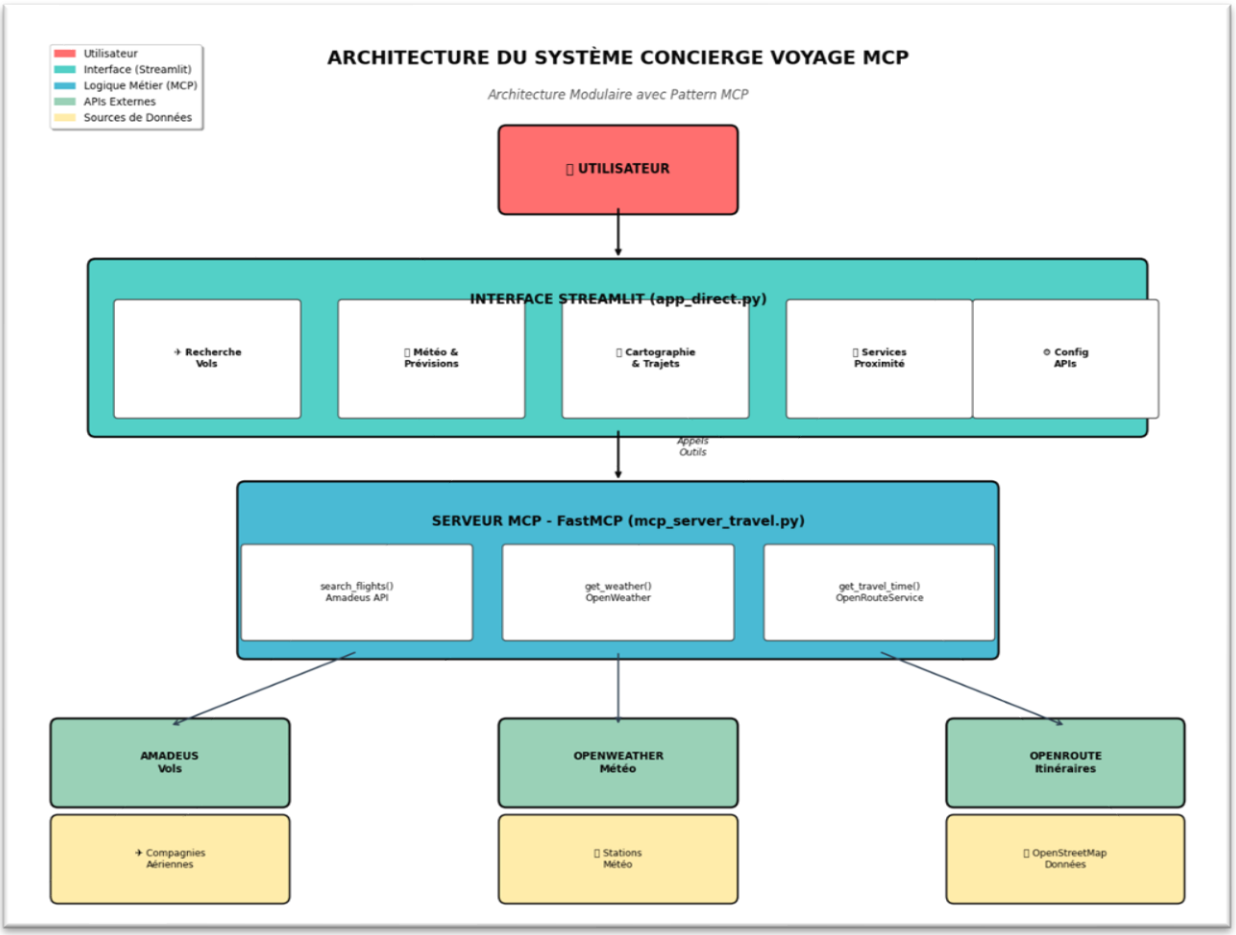

FIGURE 1 – Architecture en couches du système Concierge Voyage MCP

3.1.1 Description des Couches

Couche Présentation (Streamlit UI) Interface web interactive développée avec Streamlit, offrant trois onglets fonctionnels (Vols, Météo, Trajets) et gérant l'état de session via st.session_state pour assurer la persistance des données entre les interactions.

Couche Métier (MCP Server) Cœur du système implémenté avec FastMCP, exposant trois outils principaux :

- `search_flights()` pour la recherche aérienne
- `get_weather_forecast()` pour les données météo
- `get_travel_time()` pour les itinéraires

Couche Accès Données Gère la communication avec les APIs externes via le SDK Amadeus (vols), des requêtes HTTP REST (météo, routing), et le géocodage d'adresses.

Couche Sources Externes Données brutes provenant des compagnies aériennes, des stations météorologiques mondiales, et de la base cartographique OpenStreetMap.

3.2 Flux de Données - Recherche de Vols

Le processus de recherche de vols illustre parfaitement l'interaction entre les couches :

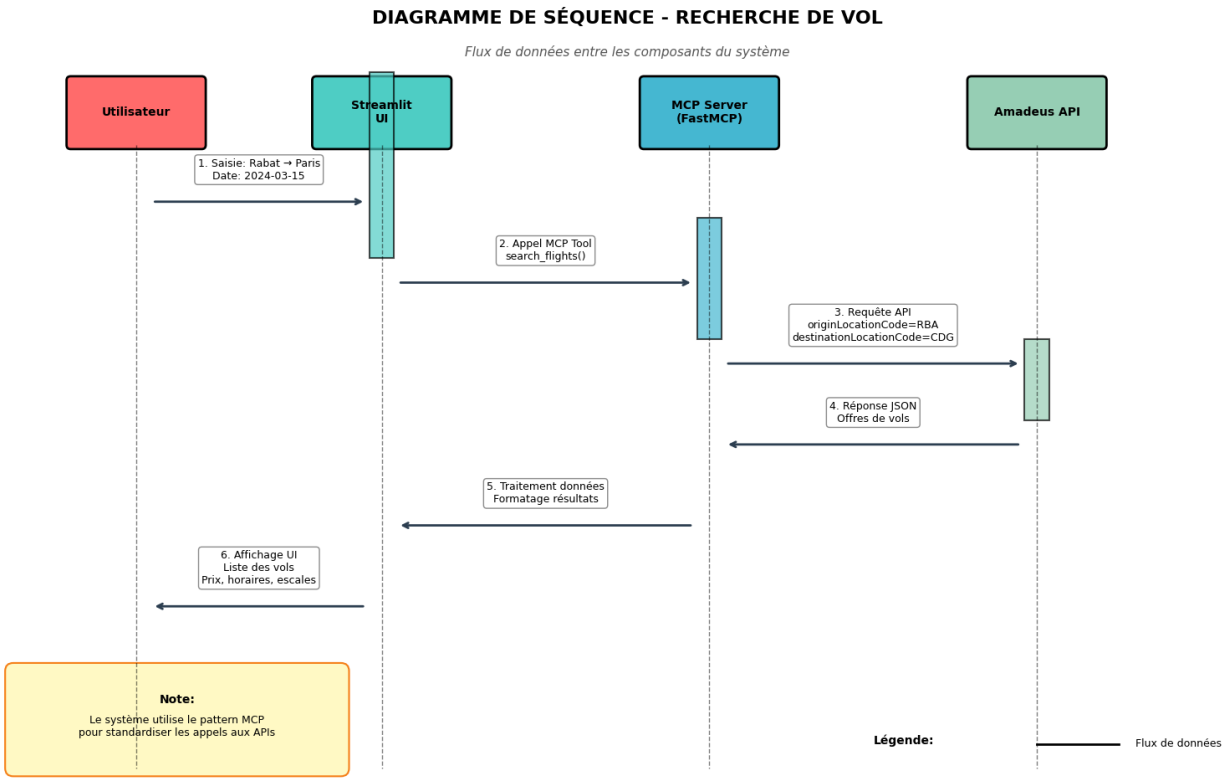

FIGURE 2 – Diagramme de séquence - Processus de recherche de vols

3.2.1 Séquence Détaillée

1. **Saisie utilisateur** : L'utilisateur entre "Rabat" → "Paris" via l'interface Streamlit
2. **Résolution IATA** : Le système convertit les noms de villes en codes aéroportuaires (RBA → CDG) via la base de données interne AIRPORTS_DB
3. **Appel MCP** : Streamlit invoque l'outil search_flights() du serveur MCP
4. **Requête Amadeus** : Le serveur formate et envoie une requête à l'API Amadeus avec les paramètres standardisés
5. **Traitement de la réponse** : Les données JSON brutes sont parsées pour extraire les prix, horaires, compagnies et durées
6. **Affichage** : Streamlit rend les résultats sous forme de cartes interactives avec métriques clés

3.3 Flux de Données - Calcul d'Itinéraire

Le calcul d'itinéraire représente le flux le plus complexe, impliquant géocodage, routing et enrichissement par les POIs :

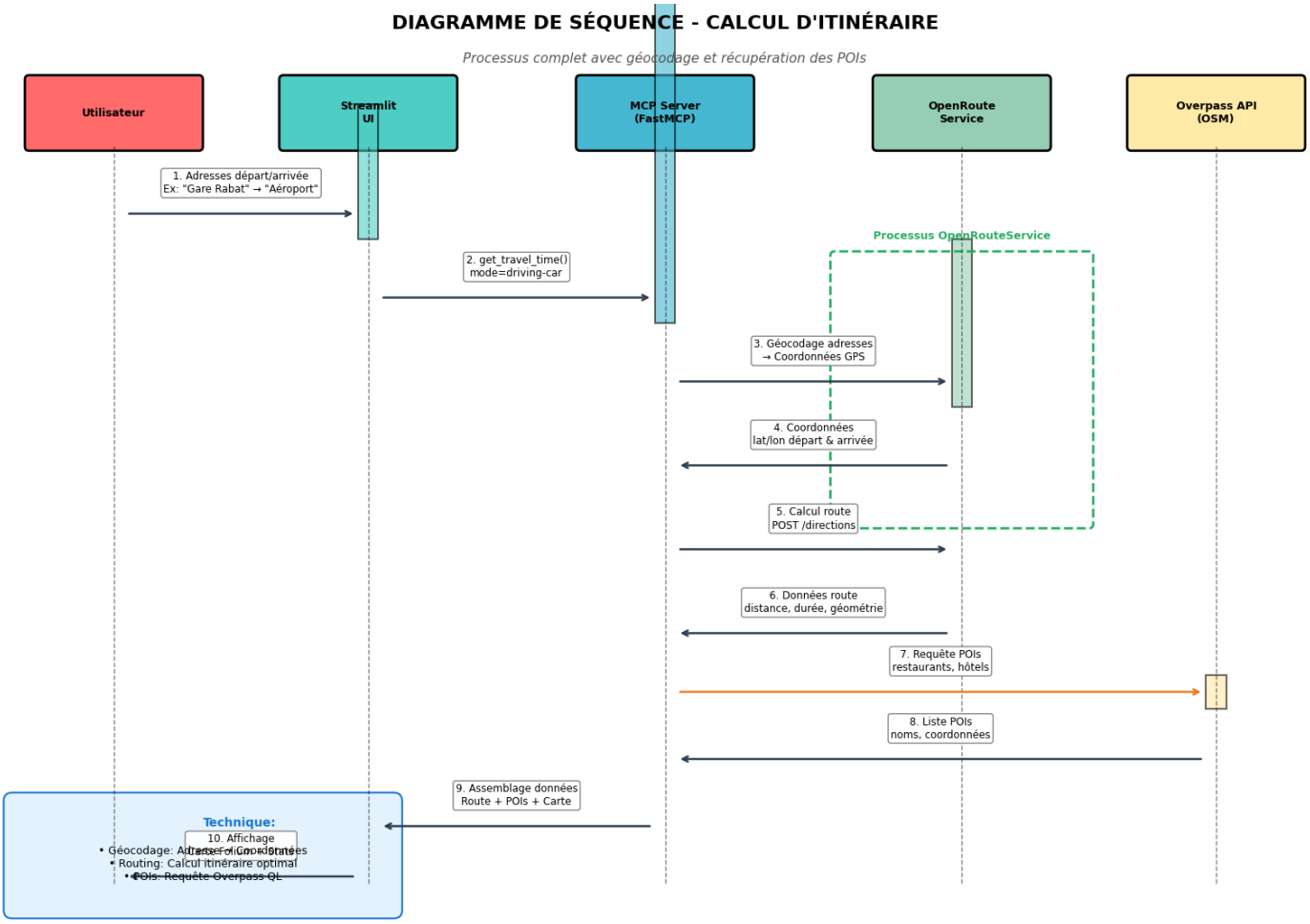

FIGURE 3 – Diagramme de séquence - Calcul d'itinéraire avec POIs

3.3.1 Processus Technique

Géocodage bidirectionnel Les adresses textuelles sont converties en coordonnées GPS (lat, lon) via l'API OpenRouteService

Calcul de route Requête POST à l'endpoint /directions avec les coordonnées et le mode de transport

Extraction de la géométrie La polyligne du trajet est récupérée au format GeoJSON et convertie pour Folium

Enrichissement POI Requête simultanée à l'API Overpass pour récupérer les restaurants, hôtels et cafés dans un rayon de 1km

Composition Agrégation des données de route, statistiques, géométrie pour la carte, et POIs pour les marqueurs

Rendu cartographique Génération d'une carte Folium interactive

3.4 Interactions entre Composants

Le système utilise un **pattern de fallback intelligent** pour garantir la résilience :

TABLE 1 – Stratégies de Fallback par Service

Service Principal	Fallback	Déclencheur
OpenWeather	Open-Meteo (gratuit)	Timeout ou erreur HTTP
OpenRouteService	OSRM (Open Source)	Quota dépassé ou indisponibilité
Amadeus	Message utilisateur détaillé	Erreur d'authentification ou paramètres invalides

La **persistance d'état** est assurée par st.session_state qui conserve :

- Les aéroports sélectionnés (origin_selected, dest_selected)
- Les dernières données de recherche (last_search)
- Les préférences utilisateur entre les changements d'onglets

4 Axe 2 : Interfaces Utilisateur

L'interface utilisateur suit une **organisation tabulaire** à trois volets, conçue pour une expérience mobile-first :

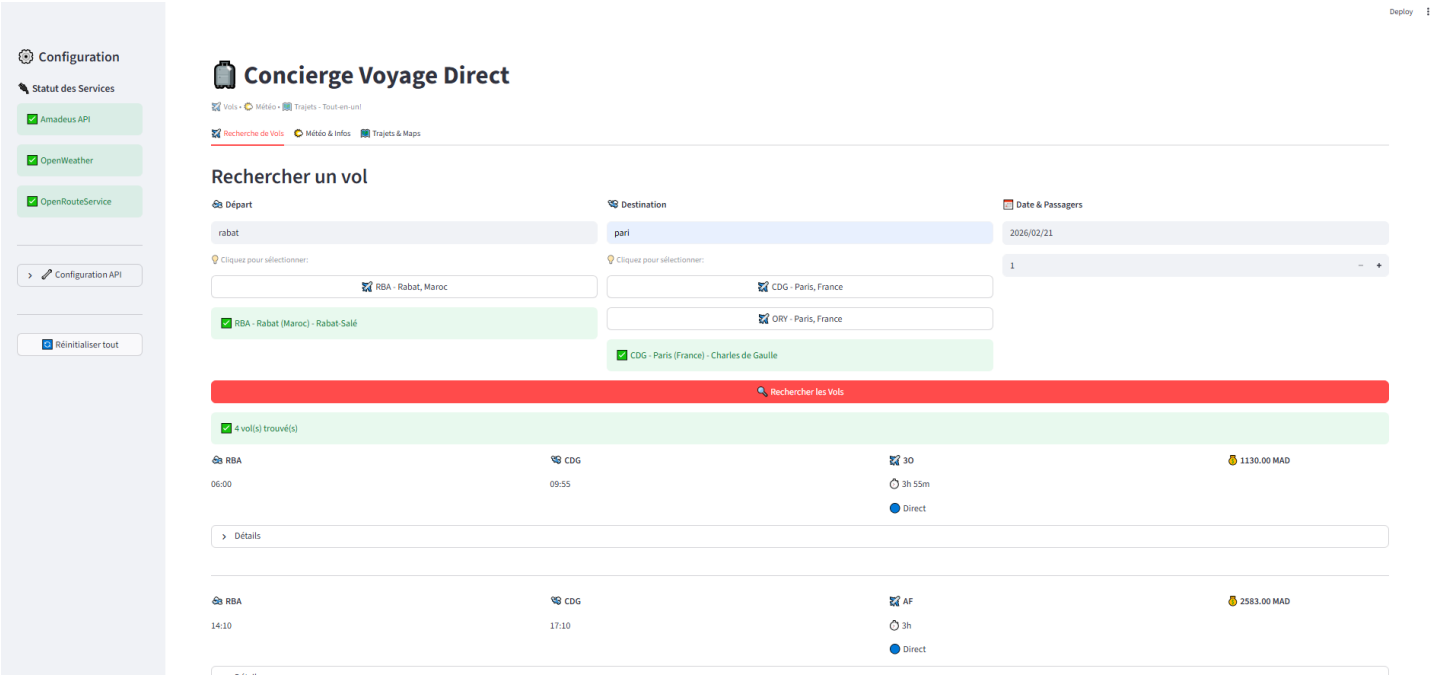

FIGURE 4 – Mockups des interfaces utilisateur - Les trois onglets principaux

4.1 Onglet "Recherche de Vols" ()

4.1.1 Composants Clés

Champs de saisie intelligente Autocomplétion avec suggestions d'aéroports basées sur une recherche floue (score de matching sur code IATA, nom de ville, mots-clés).

Sélecteur de date Contrainte min_value=date.today() pour empêcher les recherches dans le passé.

Cartes de résultats Affichage en grille responsive avec 4 colonnes :

- Colonne 1 : Aéroport et heure de départ ()
- Colonne 2 : Aéroport et heure d'arrivée ()

— Colonne 3 : Compagnie, durée formatée, indicateur d'escale
— Colonne 4 : Prix en MAD avec mise en évidence visuelle

Interaction Les boutons de suggestion mettent à jour st.session_state et déclenchent st.rerun() pour rafraîchir l'interface immédiatement.

4.2 Onglet "Météo & Infos" ()

4.2.1 Architecture

Cartes météo jumelles Deux conteneurs stylisés avec dégradés de couleurs distincts (bleu pour le départ, rouge pour la destination).

Données affichées

— Température actuelle et ressenti
— Icône météo dynamique (via URL OpenWeather)
— Humidité, visibilité, vitesse du vent, couverture nuageuse

Prévisions 5 jours Grille de 5 colonnes affichant pour chaque jour : date, icône, température, description courte.

4.2.2 Intelligence Contextuelle

Si une recherche de vol a été effectuée dans l'onglet précédent, les aéroports sont automatiquement pré-sélectionnés via st.session_state.last_search.

4.3 Onglet "Trajets & Maps" ()

4.3.1 Fonctionnalités Avancées

Mode de trajet Radio buttons pour choisir entre :

— "Depuis l'aéroport" (vers centre-ville)
— "Vers l'aéroport" (depuis centre-ville)
— "Trajet personnalisé" (adresses libres)

Carte interactive Folium

— Polyligne bleue représentant le trajet calculé
— Marqueurs personnalisés (vert pour départ, rouge pour arrivée)
— Marqueurs POIs avec icônes FontAwesome

Statistiques de trajet Métriques clés affichées : distance en km, durée en minutes, nombre d'étapes.

5 Axe 3 : Technologies et Concepts Architecturaux

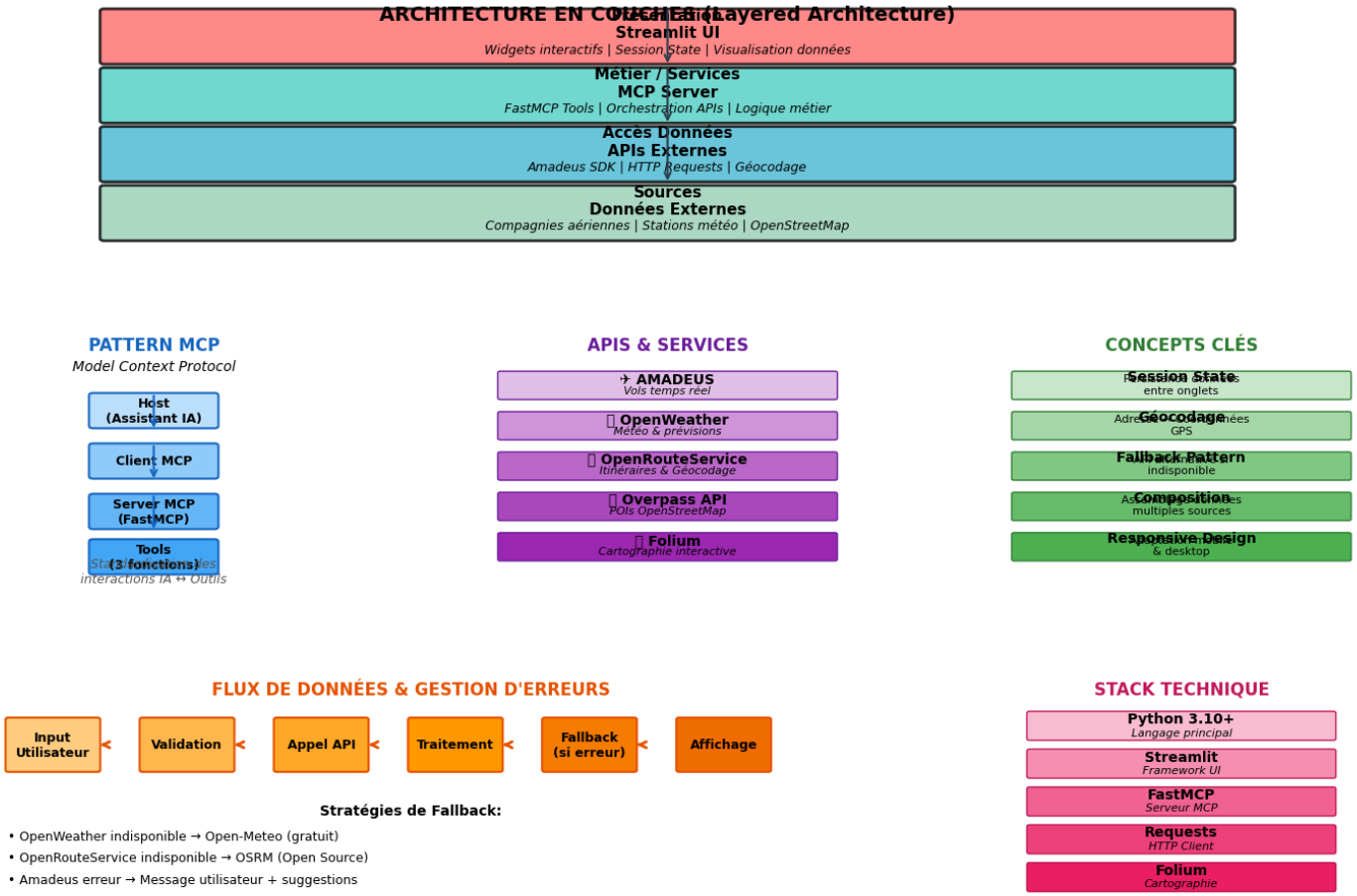

FIGURE 5 – Stack technique et concepts architecturaux

5.1 Stack Technique Complet

TABLE 2 – Stack Technique Détaillé

Couche	Technologie	Version	Rôle
Langage	Python	3.10+	Logique métier, orchestration
Framework UI	Streamlit	≥1.28.0	Interface web interactive
Serveur MCP	FastMCP	≥0.4.0	Exposition des outils MCP
SDK Vols	Amadeus	≥8.0.0	Intégration API aérienne
Cartographie	Folium	≥0.14.0	Génération cartes interactives
Bridge	streamlit-folium	≥0.15.0	Intégration Folium dans Streamlit
HTTP Client	Requests	≥2.31.0	Appels APIs REST
Env	python-dotenv	≥1.0.0	Variables d'environnement

5.2 Pattern MCP (Model Context Protocol)

Le **Model Context Protocol** est un standard émergent permettant aux assistants IA d'interagir avec des outils externes de manière sécurisée et standardisée.

5.2.1 Structure du Serveur MCP

```python
from fastmcp import FastMCP

mcp = FastMCP("ConciergeVoyage")

@mcp.tool()
def search_flights(origin: str, destination: str,
                   date: str, adults: int = 1) -> str:
    """Recherche de vols via Amadeus"""
    # Logique de recherche de vols
    pass

@mcp.tool()
def get_weather_forecast(city: str, date: str) -> str:
    """ M t  o via OpenWeather avec fallback"""
    # Logique  m  t  o avec fallback
    pass

@mcp.tool()
def get_travel_time(origin: str, destination: str,
                    mode: str = "driving-car") -> str:
    """Itinraires via OpenRouteService"""
    # Logique d'itinraire avec fallback
    pass
```

Listing 2 – Définition des outils MCP (mcp_server_travel.py)

5.2.2 Avantages du Pattern MCP

- **Standardisation** : Interface uniforme pour tous les outils (entrées/sorties JSON)
- **Découplage** : Les outils peuvent être utilisés indépendamment de l'interface
- **Interopérabilité** : Compatible avec Claude Desktop, Continue, et autres clients MCP
- **Sécurité** : Exécution sandboxée des outils avec validation des entrées

5.3 APIs Externes Intégrées

5.3.1 Amadeus API (Vols)

Type SDK Python officiel + REST API
Authentification OAuth 2.0 (client_id + client_secret)
Endpoints shopping.flight_offers_search.get()
Données Offres de vols avec prix, compagnies, horaires, escales

5.3.2 OpenWeather API (Météo)

Endpoints /data/2.5/weather (actuel), /data/2.5/forecast (prévisions)
Fallback Open-Meteo API (gratuite, sans clé)

5.3.3 OpenRouteService (Itinéraires)

Services /geocode/search, /v2/directions/{mode}/geojson
Modes driving-car, foot-walking, cycling-regular
Fallback OSRM + Nominatim

5.3.4 Overpass API (POIs)

```
[out:json];
(
  node["amenity"="restaurant"](around:{radius},{lat},{lon});
  node["amenity"="cafe"](around:{radius},{lat},{lon});
  node["tourism"="hotel"](around:{radius},{lat},{lon});
);
out body {limit};
```

Listing 3 – Requête Overpass QL pour les POIs

5.4 Concepts Architecturaux Clés

5.4.1 Session State Management

Streamlit étant sans état (stateless) par défaut, le système utilise st.session_state comme store de données côté client pour la persistance des sélections entre les onglets.

5.4.2 Pattern Fallback

```python
def get_weather_forecast(city: str, date: str) -> str:
    try:
        # Tentative principale avec OpenWeather
        return _fetch_openweather(city, date)
    except Exception:
        # Fallback sur Open-Meteo
        return _fallback_weather(city, date)
```

Listing 4 – Implémentation du pattern Fallback

5.4.3 Composition de Données

Le système agrège des données de sources hétérogènes :

— **Vol complet** = Données Amadeus + Météo départ + Météo arrivée + Trajet aéroport-centre-ville

— **Itinéraire enrichi** = Route ORS + POIs Overpass + Calcul de distance/durée

5.5 Gestion des Erreurs et Sécurité

5.5.1 Stratégies de Gestion d'Erreurs

1. **Validation des entrées** : Vérification des codes IATA, formats de date
2. **Try/except granulaire** : Chaque appel API encapsulé avec message spécifique
3. **Fallback hiérarchique** : API premium → API gratuite → Message utilisateur

5.5.2 Sécurité

— **Variables d'environnement** : Clés API stockées dans .env

— **Secrets Streamlit** : Support de st.secrets pour le déploiement cloud

— **Pas de données sensibles** : Aucune donnée utilisateur persistée côté serveur

6 Conclusion

Concierge Voyage MCP représente une **architecture moderne et évolutive** répondant aux défis de l'information voyageur fragmentée. En adoptant le pattern MCP, le système se positionne à l'intersection de l'interface utilisateur traditionnelle et de l'ère des agents IA autonomes.

6.1 Forces Techniques

— **Modularité** : Chaque outil MCP est indépendant et testable unitairement

— **Résilience** : Fonctionnement continu malgré les défaillances partielles des APIs

— **Extensibilité** : Ajout de fonctionnalités suivant le même pattern

— **Maintenabilité** : Séparation claire des responsabilités

6.2 Perspectives d'Évolution

- **Cache Redis** : Réduction des appels API
- **Base de données utilisateur** : Sauvegarde des recherches favorites
- **Exposition SSE** : Server-Sent Events pour clients distants
- **Version mobile native** : Utilisant les mêmes outils MCP via API REST